

platform

Brok PRD: Perplexity-Style AI Answer Engine

1. Product Vision

Brok is a Perplexity-style AI search engine where users can ask questions and get direct answers with sources, citations, follow-up suggestions, saved threads, and a clean research workflow.

The goal is not to feel like a normal chatbot.

The goal is:

Google search + AI answer + citations + follow-up research + memory + clean workspace

Brok should feel like:

Ask anything → Brok searches → reads sources → thinks → answers clearly → suggests next questions

The core product experience should be fast, minimal, trustworthy, and addictive.

2. Core User Flow

Main flow

User lands on Brok

- sees one big search/chat input
- asks a question
- Brok starts searching
- live search/research status appears
- sources appear
- answer streams in
- citations attach to claims
- follow-up questions appear

- user clicks or types follow-up
- thread continues with context
- user can save/share/export

The entire product should be built around this loop:

Question → Search → Sources → Answer → Follow-ups → Deeper answer

3. Main Product Pages

/

Homepage search

/search

Main answer page

/library

Saved threads and collections

/discover

Trending topics / suggested research

/spaces

Project-specific research spaces

/settings

Account, model, billing, preferences

/admin

Your internal admin panel

For MVP, focus on:

/

search page

thread history

source citations

follow-ups

basic library

4. Homepage PRD

Goal

Make the homepage feel extremely simple and powerful.

It should have almost nothing except the Brok brand, input box, and a few example prompts.

Layout

Top nav:

- Brok logo
- Discover
- Library
- API
- Sign in / Profile

Center:

- Big Brok logo or wordmark
- Search input
- Mode selector
- Example prompts

Bottom:

- Small trending/recent queries

Main input

Placeholder ideas:

Ask anything...

Research anything...

What do you want to understand?

Ask Brok anything

Input should support:

text question

file upload later
URL paste
image upload later
voice later

Input actions

Inside the input:

+ attach
model/mode selector
submit button

Modes:

Search
Deep Research
Code
Quick Answer

For MVP:

Search
Deep Search

Example prompts

What are the best AI browsers right now?
Explain how OpenAI-compatible APIs work
Compare Perplexity, ChatGPT, and Claude
Find recent YC companies building AI agents
Summarize the latest MiniMax model pricing

5. Search / Answer Page PRD

This is the most important page.

Page route

/search?q=...

or thread route:

/t/thread_id

Layout

Left sidebar:

- New thread
- Recent threads
- Library
- Spaces
- Settings

Main content:

- User question
- Search/research progress
- Sources
- Streaming answer
- Follow-up questions
- Follow-up input

Right sidebar:

- Source list
- Related topics
- Thread tools

For MVP, keep it simpler:

Left sidebar

Main answer area

Sources shown inline or above answer

6. Answer Generation Flow

When user asks:

"what are the best AI browsers?"

Brok should not immediately call MiniMax only.

It should run a search pipeline.

Backend flow

1. Receive user question.
2. Classify question type.
3. Decide whether search is needed.
4. Generate search queries.
5. Search web.
6. Fetch top sources.
7. Extract clean text.
8. Rank sources.
9. Compress context.
10. Send source context + user question to MiniMax.
11. Stream answer.
12. Attach citations.
13. Generate follow-up questions.
14. Save thread.
15. Save usage.

Question classification

Brok should classify every question:

fresh/current
evergreen/explainer
comparison
recommendation
technical
academic
shopping-ish
local
news
code
opinion

Examples:

"what is DNS?" → evergreen, no web required or light web

"latest AI browsers" → current, web required

"compare MiniMax and OpenAI pricing" → current/comparison, web required

"explain TCP congestion control" → technical explainer, maybe no web

7. Search Modes

Brok should support multiple answer modes.

Quick Answer

Fast, low-cost, fewer sources.

3 sources

short answer

fast

cheap

Use for:

simple questions

definitions

quick comparisons

basic explanations

Search

Default mode.

5-8 sources

citations

balanced answer

normal speed

Use for most queries.

Deep Search

More serious research.

10-20 sources
multiple search queries
source deduplication
more detailed answer
better citations
slower

Use for:

market research
startup research
technical comparisons
legal/financial-ish summaries
product strategy
academic topics

Code

Powered by OpenCode + MiniMax.

understands repo/code snippets
explains code
generates implementation plans
debugs

8. Search Progress UX

This is where Brok should feel alive.

After user submits a question, show a live progress module.

Example:

Searching the web...
Found 12 relevant sources

Reading product pages...
Comparing current pricing...
Building answer...

Do not expose fake chain-of-thought. Show useful task status.

Progress states

Understanding question
Searching web
Reading sources
Checking source quality
Extracting key facts
Writing answer
Adding citations
Generating follow-ups

Visual style

Each step can appear as a small animated row:

- ✓ Understanding question
- ✓ Searching 8 sources
- ✓ Reading 5 pages
- Writing answer

This creates the Perplexity feeling that the system is actively researching.

9. Sources PRD

Sources are the trust layer.

Source card

Each source card should show:

source number

title
domain
favicon
short snippet
date if available
open link

Example:

[1] MiniMax API Pricing
platform.minimax.io
Official pricing page for MiniMax models...

Source placement

Show sources in two places:

Top source strip

Immediately above the answer:

Sources
[1] MiniMax Docs [2] Reuters [3] GitHub [4] Blog

Inline citations

Inside answer:

MiniMax M2.7 is priced at...

User should be able to click citation numbers.

Source side panel

When citation is clicked:

Right panel opens
Shows source title
URL
Relevant excerpt

Why Brok used this source
Open original

Source quality logic

Prioritize:

official docs
primary sources
reputable news
academic papers
company blogs
documentation

Deprioritize:

SEO spam
content farms
random blogs
duplicate syndicated articles
low-quality AI slop

10. Answer Format PRD

Answers should be clear, direct, and structured.

Default answer structure

Direct answer
Key points
Details
Sources/citations
Follow-up suggestions

For simple questions:

Short answer first
Then explanation

For comparisons:

- Summary
- Table
- Recommendation
- Details

For technical questions:

- Concept
- How it works
- Example
- Common mistakes

For startup/product questions:

- Hard truth
- Product logic
- User flow
- Implementation
- Risks
- Next steps

Since you like product strategy, Brok can have a “builder mode” later that gives more founder-style answers.

11. Follow-Up Questions PRD

Follow-ups are central to the Perplexity-style loop.

Goal

After every answer, Brok should suggest useful next questions that help the user continue research.

Placement

At the bottom of every answer:

Related

- How does MiniMax compare to Claude for coding agents?
- What would the architecture for this look like?
- How much would this cost at 10,000 users?

Follow-up behavior

When user clicks a follow-up:

1. It becomes the next user message.
2. Brok uses thread context.
3. It decides whether new search is needed.
4. It may reuse previous sources.
5. It searches again if freshness is needed.
6. It answers as part of the same thread.

Follow-up generation logic

Generate follow-ups based on:

original question
answer content
unanswered areas
user intent
search results
thread history

Follow-up types

Each answer should generate 3-5 follow-ups.

Types:

deeper explanation
comparison
implementation
cost/pricing
risks

examples
step-by-step
current/latest

Example for user asking about Brok:

Related

- What should Brok's homepage look like?
- How should Brok generate citations?
- What should the API playground include?
- How do we make Brok feel faster than Perplexity?
- What is the MVP build plan?

Follow-up UX details

Follow-up chips should be:

short
clickable
context-aware
not generic

Bad follow-up:

Tell me more

Good follow-up:

Design the Brok answer page UI

12. Thread System PRD

Every user query creates or continues a thread.

Thread data

Each thread stores:

thread_id
user_id
title
messages
sources
citations
follow_ups
created_at
updated_at
mode

Auto-title

Brok should auto-generate short thread titles.

Examples:

MiniMax API Pricing
Brok Product Architecture
AI Browser Comparison
CSE 150 TCP Notes

Thread continuation

When user asks follow-up:

Use previous question
Use previous answer summary
Use prior sources if relevant
Search again if needed

Do not resend the entire thread blindly forever. Compress it.

Thread memory

Store:

short thread summary
important entities
source list

user intent
open questions

This helps future follow-ups stay coherent without expensive context.

13. Follow-Up Context Logic

This part matters.

When a user asks a follow-up like:

"how would the API work?"

Brok must understand what "the API" means from previous context.

Follow-up resolver

Before answering, run a resolver:

Current follow-up: "how would the API work?"

Thread context: Brok / Perplexity clone / MiniMax / Sonar API

Resolved query: "How would the Brok Sonar-style API work using MiniMax as backend?"

Search decision

Then decide:

Does this need web search?

Examples:

"explain more simply" → no search

"what is latest pricing?" → search

"build the PRD" → no search, use context

"compare to current Perplexity" → search

Follow-up answer should mention context

Example:

For Brok, the API should work like this...

Not:

An API generally works like this...

14. Query Rewriting PRD

Brok needs query rewriting before search.

Why

Users ask messy questions:

"how does this compare to sonar"

"what abt pricing"

"make it like perplexity"

Brok should rewrite them into better search queries.

Rewrite examples

User:

"what abt pricing"

Context:

MiniMax API, Brok API, Perplexity-style clone

Rewritten search queries:

MiniMax API pricing 2026

MiniMax M2.7 token pricing
Perplexity Sonar API pricing

Query rewrite output

```
{  
  "resolved_user_intent": "Compare MiniMax API pricing with Perplexity Sonar-style API pricing",  
  "needs_search": true,  
  "search_queries": [  
    "MiniMax API pricing M2.7",  
    "Perplexity Sonar API pricing",  
    "AI search API pricing comparison"  
  ]  
}
```

15. Search Pipeline PRD

Search pipeline steps

1. Rewrite query.
2. Generate 2-5 search queries.
3. Run search provider.
4. Get top 20 results.
5. Deduplicate by domain/title/content.
6. Score source quality.
7. Fetch top pages.
8. Extract clean article/content.
9. Chunk content.
10. Rerank chunks by relevance.
11. Compress source context.
12. Answer with citations.

Source scoring

Each source gets score:

relevance
authority

freshness
uniqueness
content quality
citation usefulness

Source scoring example

Official docs: +40
Recent article: +20
Exact query match: +20
Spammy domain: -50
Duplicate content: -30
Old article for current topic: -20

16. Citation Engine PRD

Goal

Every important factual claim should be tied to a source.

Citation behavior

The model receives context chunks with IDs:

[source_1]
MiniMax pricing page says...

[source_2]
Reuters article says...

The answer should cite like:

MiniMax's output tokens are more expensive than input tokens [1].

Citation rules

Cite pricing

Cite current facts
Cite company claims
Cite news
Cite statistics
Cite direct comparisons

No citation needed for:

general reasoning
product recommendations
your own architecture suggestions
UX opinions

Citation click behavior

Clicking [1] opens:

source title
domain
snippet used
full URL
open source button

17. Related Questions System

This is different from follow-ups but similar.

Follow-ups

Generated for the current answer.

Related questions

Broader suggestions based on the topic.

Example:

For “MiniMax API pricing”:

Follow-ups:

How much would 10,000 Brok users cost?

How do I add rate limits?

How do I prevent API abuse?

Related questions:

What is Perplexity Sonar?

What are OpenAI-compatible APIs?

What is prompt caching?

MVP can merge both into one section called:

Related

18. Brok Chat UI PRD

Visual layout

Sidebar:

- New Search
- Recent
- Library
- Spaces
- API
- Settings

Main:

- Question
- Research progress
- Sources
- Answer
- Follow-ups
- Chat input

Top:

- Model/mode selector
- Share

- Save

Answer page components

UserQuestion
ResearchProgress
SourceStrip
AnswerRenderer
CitationPopover
FollowUpChips
FollowUpInput
ThreadActions

Follow-up input

At bottom:

Ask a follow-up...

It should stay sticky while scrolling.

Thread actions

Share
Save
Copy
Export
Regenerate
Change mode

19. Brok Answer Interaction Features

Regenerate

User can regenerate answer with:

same sources

new sources
shorter
longer
more technical
simpler

Rewrite answer

Options:

make shorter
make deeper
explain like I'm 12
turn into table
turn into PRD
turn into steps

Source controls

User can:

open source
remove source
search more sources
ask about this source

Ask about source

When viewing a source, user can ask:

Summarize this source
What does this source say about pricing?
Is this source reliable?

20. Spaces PRD

Spaces are project-specific research folders.

For example:

Brok Startup
CSE 150
EB1A Research
Capy
Chem Lab

Each Space has:

threads
saved sources
uploaded files
custom instructions
memory

For Brok, this is powerful because users can keep research organized.

Space flow

User creates Space: "Brok"
Adds threads
Adds docs
Asks questions inside Space
Brok uses Space context

Example:

In Brok Space:
"make the homepage PRD"

Brok knows:

Brok is Perplexity-style search app + API gateway + MiniMax backend

21. Library PRD

The Library is where saved threads live.

Library layout

Search bar

Filters

Thread cards

Collections

Saved sources

Filters

All

Saved

Shared

API

Search

Code

Deep Research

Thread card

Shows:

title

summary

date

sources count

mode

last updated

22. Discover PRD

Discover is optional for MVP but useful later.

Purpose

Make Brok feel alive even when user does not know what to ask.

Sections

Trending in AI
Trending in startups
Trending in tech
Suggested for you
Recent research from your spaces

Example cards

AI browsers are heating up
MiniMax releases new model pricing
YC startups building coding agents
Open-source alternatives to Perplexity

This can help retention.

23. Answer Quality System

Brok should optimize for:

accuracy
source quality
clarity
speed
usefulness
citation coverage

Quality checks before final answer

Before final response is complete, backend should check:

Does answer address user question?
Are citations attached?
Are sources relevant?
Is answer too vague?

Is it too long?
Does it include unsupported claims?

For MVP, this can be prompt-based.

Later, add evaluator model.

24. User Personalization

Brok should learn user preferences over time.

Examples:

User prefers PRDs
User likes product strategy
User likes simple explanations
User is building Brok
User wants founder-style answers

For Brok specifically, store project memory inside Spaces.

Personalization examples

If user asks:

"make this better"

Brok knows whether to make it:

more startup-like
more technical
more polished
more simple

25. Brok Search Result Types

Brok should support different answer templates.

Explainer

For questions like:

What is statistical multiplexing?

Output:

Simple answer

Analogy

Step-by-step

Example

Follow-ups

Current research

For:

Latest AI browser startups

Output:

Summary

Key updates

Companies

Why it matters

Sources

Follow-ups

Comparison

For:

MiniMax vs OpenAI for agents

Output:

Quick verdict
Comparison table
Best use cases
Cost notes
Recommendation

PRD / product

For:

Make me a PRD

Output:

Vision
User flow
Pages
Features
Backend
Data model
MVP plan

Technical

For:

How do I build API keys?

Output:

Architecture
Request flow
Security
Schema
Implementation plan

26. Follow-Up Examples

User asks:

"what is minimax and why is it cheap?"

Brok answer ends with:

Related

- How much would MiniMax cost for 10,000 Brok users?
- How should Brok route requests through MiniMax?
- What rate limits should Brok add?
- How does MiniMax compare to OpenAI for coding agents?

User asks:

"make me a Perplexity clone PRD"

Brok answer ends with:

Related

- Design the Brok search page UI
- Write the backend search pipeline
- Create the Brok citation system
- Add Spaces and saved research
- Build the MVP sprint plan

User asks:

"how do follow-ups work?"

Brok answer ends with:

Related

- Build the follow-up generation prompt
- Design the follow-up UI chips
- Explain how thread context is compressed
- Show the database schema for threads

27. Follow-Up Generation Prompt Logic

After answer is generated, run a small follow-up generator.

Input:

user question
final answer summary
thread context
mode
user profile

Output:

```
{
  "follow_ups": [
    {
      "label": "Design the Brok homepage UI",
      "query": "Create a detailed UI spec for the Brok homepage"
    },
    {
      "label": "Build the citation system",
      "query": "Explain how Brok should generate and display citations"
    },
    {
      "label": "Plan the MVP backend",
      "query": "Give me the backend architecture for Brok MVP"
    }
  ]
}
```

Rules

Follow-ups should be:

specific
useful
short
context-aware

actionable

Avoid:

Would you like to know more?

Can I help with anything else?

Tell me more

28. Thread Context Compression

Brok cannot send the full thread forever. It needs compression.

Store per thread

thread_summary

key_entities

user_goal

important_decisions

sources_used

open_questions

Example:

```
{
  "thread_summary": "User is designing Brok, a Perplexity-style AI answer engine and API platform powered by MiniMax and OpenCode.",
  "key_entities": ["Brok", "MiniMax", "OpenCode", "Perplexity", "Sonar-style API"],
  "user_goal": "Build PRDs and product architecture for Brok",
  "important_decisions": [
    "Rename everything Brok",
    "Use Brok API keys",
    "Add API playground to admin panel"
  ],
  "open_questions": [
    "How should follow-ups work?",
    "What should the search UI look like?"
  ]
}
```

When user says:

"add follow ups"

Brok resolves:

Add follow-up question functionality to the Brok Perplexity-style answer engine.

29. Database Schema

threads

threads

- id
- user_id
- space_id
- title
- summary
- mode
- created_at
- updated_at

messages

messages

- id
- thread_id
- role
- content
- resolved_query
- mode
- created_at

answers

answers

- id
- message_id

- content
- model
- search_mode
- input_tokens
- output_tokens
- latency_ms
- created_at

sources

sources

- id
- thread_id
- answer_id
- title
- url
- domain
- snippet
- published_at
- retrieved_at
- quality_score

citations

citations

- id
- answer_id
- source_id
- claim_text
- citation_number
- source_excerpt

follow_ups

follow_ups

- id
- answer_id
- label
- query
- clicked
- created_at

spaces

spaces

- id
 - user_id
 - name
 - description
 - instructions
 - created_at
-

30. API Endpoints

Create thread/search

POST /api/search

Request:

```
{  
  "query": "make me a PRD for a Perplexity clone",  
  "mode": "search",  
  "space_id": null  
}
```

Response:

```
{  
  "thread_id": "thr_123",  
  "message_id": "msg_123",  
  "stream_url": "/api/search/stream/msg_123"  
}
```

Stream answer

GET /api/search/stream/:message_id

Events:

query_resolved
search_started
source_found
source_read
answer_delta
citation_added
follow_ups_generated
done

Continue thread

POST /api/threads/:thread_id/messages

Request:

```
{  
  "content": "now add follow-ups",  
  "mode": "search"  
}
```

Get thread

GET /api/threads/:thread_id

Save thread

POST /api/threads/:thread_id/save

Get follow-ups

GET /api/answers/:answer_id/follow-ups

31. Streaming Event Design

To make Brok feel like Perplexity, use streaming.

Example events:

event: status

data: {"message": "Understanding your question"}

event: query

data: {"resolved_query": "Perplexity-style AI answer engine PRD with follow-ups"}

event: status

data: {"message": "Searching the web"}

event: source

data: {"title": "...", "url": "...", "domain": "..."}
}

event: status

data: {"message": "Writing answer"}

event: answer_delta

data: {"text": "Brok should..."}

event: citation

data: {"source_id": "src_1", "citation_number": 1}

event: follow_ups

data: {"items": [...]}

event: done

data: {"usage": {...}}

Frontend renders these live.

32. MVP UI Components

Components

SearchInput

ModeSelector

ThreadSidebar

ResearchProgress
SourceStrip
SourceCard
AnswerRenderer
CitationButton
CitationPanel
FollowUpChips
FollowUpInput
ThreadToolbar

Minimum MVP

Build only:

Homepage input
Answer page
Streaming answer
Sources list
Follow-up chips
Thread history

Do not overbuild Discover, Spaces, or Library immediately.

33. Brok Visual Style

Feel

minimal
fast
clean
high-trust
slightly playful
more founder/productivity than academic

UI inspiration

Perplexity for answer layout

Linear for sidebar/product polish
Stripe for docs/API trust
Raycast for speed

Colors

Since the name is **Brok**, you can do:

dark background
soft white cards
electric blue/purple accent
or black/cream minimal

Avoid clutter. The input and answer should be the star.

34. MVP Build Plan

Week 1: Search chat foundation

Homepage
Thread page
Chat input
MiniMax response streaming
Thread storage

Week 2: Search pipeline

Search API integration
Source fetching
Source ranking
Context compression
Citations

Week 3: Follow-ups

Follow-up generator

Follow-up chips
Thread continuation
Resolved query logic
Context compression

Week 4: Library + polish

Recent threads
Saved threads
Share link
Better source panel
Usage tracking
Admin logs

35. The Most Important Product Details

If you want Brok to feel like Perplexity, nail these:

1. The answer must start fast

Even while search is happening, show useful progress.

Searching...
Reading sources...
Found 6 sources...

2. Sources must feel trustworthy

Use clean source cards and inline citations.

3. Follow-ups must be good

Do not generate generic garbage. Make them feel like the next smart step.

4. Thread context must work

When user says “make it better,” Brok should know what “it” is.

5. Search should happen only when needed

Not every follow-up needs web search.

6. The UI must stay uncluttered

Perplexity works because the page is mostly answer, not buttons.

36. Final Brok Product Definition

Brok is a Perplexity-style answer engine that turns messy questions into sourced, useful answers.

It searches the web, reads sources, streams clear responses, cites claims, suggests smart follow-ups, and saves research into organized threads.

The core loop is simple:

Ask → Brok researches → answer with sources → click follow-up → go deeper.

That is the actual product.

api

Below is the full PRD with everything renamed to **Brok**.

Brok PRD: Perplexity-style Chat + Sonar-style API + Admin Usage Platform

1. Product Summary

Brok is an AI answer engine and developer API platform.

It has two main products:

1. Brok Chat

A Perplexity-style search/chat interface powered by MiniMax, web search, citations, streaming responses, and saved threads.

2. Brok API

A Sonar-style developer platform where users can create **Brok API keys**, test them in an API playground, view usage, set limits, and call Brok's endpoints.

Internally, Brok can use MiniMax, search providers, scraping tools, rerankers, and other LLMs. Externally, users only interact with:

- Brok API
- Brok API Keys
- Brok Models
- Brok Usage Dashboard
- Brok Playground

Important framing:

Brok is not “reselling a hidden MiniMax key.”

Brok is a branded AI API gateway with its own authentication, routing, search, citations, usage tracking, billing, logs, and developer experience.

2. Core Product Concept

User-facing idea

A user signs up, creates a Brok API key, and can immediately call:

POST `https://api.brok.ai/v1/chat/completions`

They use:

Authorization: Bearer `brok_sk_...`

Brok receives the request, verifies the user, checks limits, records usage, routes the request to MiniMax or another backend model, streams the response back, and logs the full usage event.

3. Product Modules

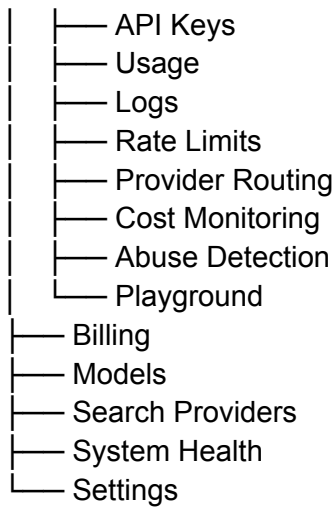
Brok should have these main areas:

- /brok-chat
- /dashboard
- /admin
- /api-keys
- /playground
- /usage
- /logs
- /billing
- /docs
- /settings
- /team

For your existing admin panel, add a new section:

Admin Panel

- Overview
- Users
- Brok API



4. Naming System

Rename everything from Balloon/Sonar to Brok.

Product names

Balloon → Brok
Balloon Chat → Brok Chat
Balloon API → Brok API
Balloon Sonar → Brok Search API
Balloon Playground → Brok Playground
Balloon Admin → Brok Admin
Balloon Keys → Brok API Keys
Balloon Models → Brok Models

Model names

brok-lite
brok-search
brok-search-pro
brok-code
brok-agent
brok-reasoning

Endpoint names

/v1/chat/completions
/v1/search/completions
/v1/agents/run
/v1/code/run
/v1/models
/v1/usage
/v1/keys

API key prefixes

brok_sk_live_...
brok_sk_test_...
brok_pk_...

Only backend/server-side usage should use secret keys.

5. What “Masking APIs” Should Mean

Do **not** build this as a deceptive MiniMax wrapper.

Build this as a legitimate **API gateway**.

Correct architecture

Developer
↓
Uses Brok API Key
↓
Brok API Gateway
↓
Auth + usage + rate limits + logging + billing
↓
Provider router
↓
MiniMax / Search Provider / Other Model
↓
Brok streams response back

What users see

They see:

- Brok API key
- Brok model names
- Brok usage
- Brok logs
- Brok docs
- Brok pricing

What users do not see

They should never see:

- MiniMax API key
- Provider billing account
- Raw provider credentials
- Internal routing logic
- Fallback provider secrets

This is normal. Stripe users do not see Visa's backend rails. Vercel users do not see AWS/GCP infra credentials. Brok can expose its own clean abstraction.

6. System Architecture

High-level system

Frontend

- Brok Chat UI
- Brok Dashboard
- Brok API Playground
- Brok Admin Panel
- Brok Docs

Backend

- API Gateway
- Auth Service
- API Key Service
- Rate Limit Service
- Usage Metering Service
- Provider Router
- Search Pipeline
- LLM Adapter Layer
- Billing Service
- Logging Service
- Admin Service

Infrastructure

- Postgres
- Redis
- Queue system
- Object storage
- Observability
- Secrets manager

Suggested stack

Frontend:

Next.js
React
Tailwind
shadcn/ui
SSE streaming

Backend:

FastAPI or Node.js/NestJS
Postgres
Redis
Prisma or SQLAlchemy
Stripe Billing
Sentry
OpenTelemetry

AI Providers:

MiniMax for chat/code/reasoning
Search API for live web
Firecrawl or similar for page extraction
Optional reranker later

Infra:

Railway / Render / Fly.io for MVP

Supabase Postgres or Neon

Upstash Redis

Vercel for frontend

7. Brok API Gateway PRD

Goal

Allow developers to call Brok using Brok-owned API keys while Brok securely routes traffic to backend AI/search providers.

User story

As a developer, I want to create a Brok API key, call Brok models, view usage, and not worry about which underlying model provider Brok uses.

Core requirements

API key creation

User can create keys from dashboard:

Name: "Production app"

Environment: live/test

Allowed models: brok-lite, brok-search, brok-code

Monthly budget: \$20

RPM limit: 60

Daily request limit: 5,000

Generated key:

brok_sk_live_xxxxxxxxx

Key should only be shown once.

After creation, dashboard only shows:

brok_sk_live_.....4f92

API key scopes

Each key should have scopes:

chat:write
search:write
code:write
agents:write
usage:read
logs:read

Example:

```
{  
  "name": "Production App",  
  "scopes": ["chat:write", "search:write"],  
  "allowed_models": ["brok-lite", "brok-search"],  
  "monthly_budget_cents": 2000  
}
```

API key restrictions

Allow optional restrictions:

Allowed domains
Allowed IPs
Environment: test/live
Max tokens per request
Max output tokens
Max requests per minute
Max requests per day
Monthly spend cap

8. API Request Flow

Standard request flow

1. User sends request to Brok endpoint.
2. Brok extracts API key.
3. Brok hashes API key and looks it up.
4. Brok verifies key is active.
5. Brok checks workspace subscription.
6. Brok checks model permission.
7. Brok checks rate limits.
8. Brok checks budget.
9. Brok creates request log.
10. Brok routes request to provider.
11. Provider streams response.
12. Brok transforms response into Brok format.
13. Brok records tokens/cost/latency.
14. Brok sends final response.

Example request

```
curl https://api.brok.ai/v1/chat/completions \  
-H "Authorization: Bearer brok_sk_live_xxxxx" \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "brok-search",  
  "messages": [  
    {  
      "role": "user",  
      "content": "what is the latest news about AI browsers?"  
    }  
  ],  
  "stream": true  
'
```

Example response

```
{  
  "id": "brok_resp_123",  
  "object": "chat.completion",  
  "model": "brok-search",  
  "choices": [  
    {  
      "message": {
```

```
    "role": "assistant",
    "content": "...",
  }
},
],
"citations": [
  {
    "title": "Example Source",
    "url": "https://example.com",
    "snippet": "...",
  }
],
"usage": {
  "input_tokens": 5200,
  "output_tokens": 900,
  "search_queries": 3,
  "cached_tokens": 1200,
  "billable_units": 1
}
}
```

9. Provider Key Security PRD

Goal

Keep all provider keys secure and invisible to users.

Rules

Provider keys must never be stored in:

- frontend env
- browser localStorage
- mobile app bundle
- Chrome extension
- public GitHub repo
- client-side code
- logs
- analytics tools

Provider keys should be stored in:

backend environment variables
secrets manager
encrypted database table

Internal provider config

Provider: MiniMax

Status: Active

Models:

- minimax-m2.7
- minimax-text

Internal key:

stored in secrets manager

Mapped Brok models:

- brok-lite
- brok-code
- brok-agent

Provider adapter

Create an internal adapter layer:

Brok request format

↓

Provider adapter

↓

MiniMax request format

↓

MiniMax response

↓

Brok response format

This lets you switch providers later without changing user-facing APIs.

10. Model Routing PRD

Brok model mapping

brok-lite

→ MiniMax fast/cheap model

brok-search

→ Search pipeline + MiniMax answer synthesis

brok-search-pro

→ More search results + deeper extraction + longer context

brok-code

→ OpenCode + MiniMax

brok-agent

→ Tool-using agent with browser/search/file tools

brok-reasoning

→ Stronger reasoning route, possibly higher-cost model later

Routing logic

if model == brok-lite:

route directly to MiniMax

if model == brok-search:

run search pipeline first

send retrieved context to MiniMax

if model == brok-code:

call OpenCode orchestration layer

MiniMax powers code reasoning

if model == brok-agent:

use tool planner

execute tools

summarize final result with MiniMax

Future routing

Later you can add:

fallback providers
latency-based routing
cost-based routing
quality-based routing
region-based routing
customer-specific routing

11. Brok Search API PRD

Goal

Create a Sonar-style endpoint that gives answers with web search and citations.

Endpoint

POST /v1/search/completions

Input

```
{  
  "model": "brok-search",  
  "query": "best AI browsers in 2026",  
  "depth": "standard",  
  "stream": true  
}
```

Search modes

lite
- 3 sources
- fast answer
- cheaper

standard

- 5 to 8 sources
- citations
- balanced

deep

- 10 to 20 sources
- multi-step synthesis
- slower
- more expensive

Search pipeline

1. Receive query.
2. Rewrite query for search.
3. Run web search.
4. Select top sources.
5. Fetch source pages.
6. Extract clean content.
7. Deduplicate sources.
8. Rank source relevance.
9. Compress context.
10. Send to MiniMax.
11. Generate answer.
12. Attach citations.
13. Log search cost and token cost.

Citation format

```
{  
  "citations": [  
    {  
      "id": "src_1",  
      "title": "Page title",  
      "url": "https://example.com",  
      "publisher": "Example",  
      "snippet": "Relevant text summary",  
      "retrieved_at": "2026-05-06T00:00:00Z"  
    }  
  ]  
}
```

12. Brok Code API PRD

Goal

Expose OpenCode + MiniMax as a developer-facing code agent.

Endpoint

POST /v1/code/run

User story

As a developer, I want to give Brok a coding task and let it reason, edit, test, and return a patch or explanation.

MVP input

```
{  
  "model": "brok-code",  
  "task": "Fix the login bug in this repo",  
  "repo_url": "https://github.com/user/repo",  
  "mode": "plan_only"  
}
```

Modes

plan_only
- reads task
- returns implementation plan

patch
- generates code changes
- returns diff

agent
- runs longer workflow
- can test and iterate

For MVP, start with `plan_only` and `patch`. Do not allow arbitrary unsafe execution until sandboxing is mature.

13. Brok API Playground PRD

Goal

Give developers a clean UI to test Brok models, see code snippets, inspect usage, and compare responses.

Page route

`/dashboard/playground`

or inside admin:

`/admin/brok/playground`

Layout

Top bar:

- Environment selector: Test / Live
- API key selector
- Model selector
- Stream toggle
- Run button

Left panel:

- Prompt/messages editor
- Parameters
- System message
- Search settings

Center panel:

- Live response stream
- Citations

- Tool calls
- Response metadata

Right panel:

- Code snippets
- Usage estimate
- Request JSON
- Response JSON
- Logs link

Playground modes

Chat

Search

Code

Agent

Raw API

Chat mode

Fields:

Model

System message

User message

Temperature

Max output tokens

Stream true/false

Search mode

Fields:

Model: brok-search / brok-search-pro

Search depth: lite / standard / deep

Citation style

Recency setting

Domain filters

Code mode

Fields:

Task

Repo context

Files

Mode: plan_only / patch

Agent mode

Fields:

Goal

Tools allowed

Max steps

Require confirmation before actions

Raw API mode

Advanced developer can edit JSON directly:

```
{
  "model": "brok-search",
  "messages": [
    {
      "role": "user",
      "content": "Explain Stripe Connect"
    }
  ],
  "stream": true
}
```

Playground output

Show:

Response

Citations

Token usage

Estimated cost

Latency

Provider route

Request ID
Logs
Copy response
Copy curl
Copy JavaScript
Copy Python

Code snippet generator

The playground should auto-generate snippets.

curl

```
curl https://api.brok.ai/v1/chat/completions \  
-H "Authorization: Bearer $BROK_API_KEY" \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "brok-search",  
  "messages": [{"role": "user", "content": "Explain AI browsers"}]  
'
```

JavaScript

```
const response = await fetch("https://api.brok.ai/v1/chat/completions", {  
  method: "POST",  
  headers: {  
    "Authorization": `Bearer ${process.env.BROK_API_KEY}`,  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({  
    model: "brok-search",  
    messages: [  
      { role: "user", content: "Explain AI browsers" }  
    ]  
  })  
});
```

```
const data = await response.json();  
console.log(data);
```

Python

```
import os
import requests

response = requests.post(
    "https://api.brok.ai/v1/chat/completions",
    headers={
        "Authorization": f"Bearer {os.environ['BROK_API_KEY']}",
        "Content-Type": "application/json"
    },
    json={
        "model": "brok-search",
        "messages": [
            {"role": "user", "content": "Explain AI browsers"}
        ]
    }
)

print(response.json())
```

Playground UX details

The playground should feel like Stripe + Perplexity + OpenAI playground.

Important UI details:

- Streaming text output
- Live token counter
- Live estimated cost
- Citations expandable
- Request timeline
- Copy buttons everywhere
- Save prompt
- Load previous request
- Replay request
- Compare models

14. Admin Panel Integration PRD

Goal

Add Brok API management into the existing admin panel.

New admin section

Admin → Brok API

Subpages:

- Overview
- Users
- API Keys
- Usage
- Logs
- Rate Limits
- Provider Routing
- Cost Monitor
- Abuse Detection
- Playground
- Settings

14.1 Admin Overview

Show global stats:

- Total requests today
- Total tokens today
- Total search calls today
- Provider cost today
- Revenue today
- Gross margin
- Failed requests
- Average latency
- Top users by usage
- Top models
- Abuse alerts

Cards:

Requests
Tokens
Revenue
Cost
Margin
Latency
Errors
Active API keys

Charts:

Requests over time
Cost over time
Revenue over time
Model usage split
Error rate
Latency p50 / p95 / p99

14.2 Admin Users

Table:

User
Workspace
Plan
Requests today
Cost today
Revenue today
Margin
Status
Risk score
Created date

Actions:

View usage
View logs
Suspend user
Reset quota
Change plan

Set custom rate limit
Revoke all keys

14.3 Admin API Keys

Table:

Key name
Owner
Prefix
Environment
Status
Scopes
Created
Last used
Requests today
Monthly usage

Actions:

Revoke key
Rotate key
Pause key
Edit scopes
Edit limits
View logs

14.4 Admin Usage

Filters:

Date range
User
Workspace
API key
Model
Endpoint
Status
Provider

Metrics:

Input tokens
Output tokens
Cached tokens
Search calls
Tool calls
Provider cost
Brok billed amount
Margin
Latency
Errors

14.5 Admin Logs

Every API call should have a log.

Log fields:

request_id
user_id
workspace_id
api_key_id
endpoint
model
provider
status
input_tokens
output_tokens
cached_tokens
search_calls
latency_ms
provider_cost_usd
billed_usd
error_code
created_at

Clicking a log opens:

Request JSON

Response JSON
Timeline
Provider route
Usage
Cost
Errors
Citations
Tool calls

Sensitive fields must be redacted.

Never show:

provider API keys
full user API key
secrets
passwords
auth tokens
private headers

14.6 Admin Rate Limits

Admin can create rate-limit policies.

Examples:

Free Plan:

- 20 requests/min
- 500 requests/day
- 100K tokens/day
- brok-lite only

Starter:

- 60 requests/min
- 5,000 requests/day
- 2M tokens/month

Pro:

- 300 requests/min
- 50,000 requests/day
- 20M tokens/month

Enterprise:
- custom

Admin can override per user:

User: test@company.com
RPM: 500
Daily requests: 100,000
Monthly budget: \$500
Allowed models: all

14.7 Admin Provider Routing

Admin can control how Brok maps models to providers.

Table:

Brok model
Primary provider
Fallback provider
Status
Cost per 1M input
Cost per 1M output
Average latency
Error rate

Example:

brok-lite → MiniMax M2.7
brok-search → MiniMax M2.7 + Search Provider
brok-code → OpenCode + MiniMax
brok-agent → Tool Runner + MiniMax

Actions:

Enable model
Disable model
Change provider
Set fallback

Set max context
Set max output
Set cost multiplier

14.8 Admin Cost Monitor

This is critical.

Show:

Provider cost today
Provider cost this month
Revenue today
Revenue this month
Gross margin
Margin by model
Margin by user
Users costing more than they pay
Abnormal usage spikes

Alerts:

Provider cost exceeds \$X/day
User cost exceeds plan value
Negative margin user
Error spike
Latency spike
Search cost spike

14.9 Admin Abuse Detection

Flag suspicious activity:

Too many requests per minute
Huge output token requests
Repeated failed requests
New account using high volume instantly
Many API keys created quickly
Known proxy/VPN abuse

Prompt spam
Endpoint scraping behavior

Risk score:

0-30: normal
31-70: watch
71-100: high risk

Actions:

Throttle
Pause key
Suspend workspace
Require payment method
Force captcha on dashboard
Block IP

15. Rate Limiting PRD

Goal

Prevent abuse, protect provider keys, and keep costs predictable.

Rate limit dimensions

Use multiple limits at once:

requests per minute
requests per hour
requests per day
tokens per minute
tokens per day
monthly dollar budget
max output tokens per request
max search calls per request
max concurrent requests

Limit hierarchy

Apply limits in this order:

1. Global platform limit
2. Plan limit
3. Workspace limit
4. User limit
5. API key limit
6. Endpoint limit
7. Model limit
8. IP limit

Example limits

Free

20 RPM
500 requests/day
100K tokens/day
brok-lite only
max output: 1,000 tokens
search depth: lite only

Starter

60 RPM
5,000 requests/day
2M tokens/month
max output: 2,000 tokens
brok-lite + brok-search

Pro

300 RPM
50,000 requests/day
20M tokens/month
max output: 8,000 tokens
all standard models

Enterprise

custom RPM
custom token budget
dedicated routing
custom SLAs

Rate limit response

When exceeded:

```
{
  "error": {
    "type": "rate_limit_error",
    "code": "rate_limit_exceeded",
    "message": "Rate limit exceeded for this API key.",
    "limit": "60 requests per minute",
    "retry_after_seconds": 22
  }
}
```

Headers:

X-Brok-RateLimit-Limit: 60
X-Brok-RateLimit-Remaining: 0
X-Brok-RateLimit-Reset: 1715000000
Retry-After: 22

16. Usage Metering PRD

Goal

Track every unit of cost and usage.

Usage events table

usage_events

- id
- request_id
- user_id

- workspace_id
- api_key_id
- endpoint
- model
- provider
- input_tokens
- output_tokens
- cached_tokens
- search_queries
- pages_fetched
- tool_calls
- provider_cost_usd
- billed_usd
- latency_ms
- status
- error_code
- created_at

Token accounting

Track:

raw input tokens
cached input tokens
billable input tokens
output tokens
total tokens

Cost formula

provider_cost =
(input_tokens / 1,000,000 * provider_input_price)
+ (output_tokens / 1,000,000 * provider_output_price)
+ search_cost
+ scraping_cost
+ storage_cost

Brok billed amount

billed_amount =
provider_cost * markup

or:

deduct from monthly included usage

Recommended internal margin target

70%+ margin for brok-lite

50%+ margin for brok-search

30%+ margin for brok-agent/code

Agent/code modes will naturally cost more because of loops, tools, retries, and long context.

17. Billing PRD

Plans

Free

\$0/month

100 search answers/month

500 chat requests/month

No API access or limited test API

Starter

\$9/month

3,000 API requests/month

brok-lite

basic logs

basic playground

Pro

\$29/month

15,000 API requests/month

brok-search

brok-code
advanced logs
higher rate limits

Team

\$99/month
shared workspace
team API keys
usage analytics
higher limits
priority routing

Scale

custom pricing
usage-based
custom limits
dedicated support

Billing rules

Users should have:

included monthly credits
hard cap option
auto-recharge option
usage alerts
invoice history
cost breakdown

Usage alerts

50% used
80% used
100% used
budget exceeded
unusual spike

18. Brok Docs PRD

Goal

Make it feel like a real developer platform.

Docs pages

/docs
/docs/quickstart
/docs/api-keys
/docs/chat-completions
/docs/search-completions
/docs/streaming
/docs/citations
/docs/models
/docs/errors
/docs/rate-limits
/docs/usage
/docs/billing
/docs/sdks
/docs/security

Quickstart page

Should show:

1. Create Brok account
2. Generate API key
3. Make first request
4. View usage
5. Add billing

Docs tone

Simple, clean, serious.

Example intro:

Brok gives developers a simple API for search-powered AI responses, coding agents, and low-cost intelligence. Use Brok models through OpenAI-compatible endpoints with built-in citations, usage tracking, and streaming.

19. OpenAI-Compatible API PRD

Goal

Make Brok easy to adopt.

Support OpenAI-style requests:

POST /v1/chat/completions

Input:

```
{
  "model": "brok-lite",
  "messages": [
    {
      "role": "user",
      "content": "Hello"
    }
  ],
  "stream": true
}
```

Output:

```
{
  "id": "chatcmpl_brok_123",
  "object": "chat.completion",
  "created": 1715000000,
  "model": "brok-lite",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
```

```

    "content": "Hello! How can I help?"
  },
  "finish_reason": "stop"
}
],
"usage": {
  "prompt_tokens": 12,
  "completion_tokens": 8,
  "total_tokens": 20
}
}

```

This makes Brok usable with existing OpenAI SDKs by changing:

```

baseUrl=https://api.brok.ai/v1
apiKey=brok_sk_...

```

20. Streaming PRD

Goal

Make Brok feel fast like Perplexity.

Use Server-Sent Events.

Streaming format:

```
data: {"type":"message_start","id":"brok_resp_123"}
```

```
data: {"type":"content_delta","delta":"AI"}
```

```
data: {"type":"content_delta","delta":" browsers"}
```

```
data: {"type":"citation","citation":{"title":"...", "url":"..."}}
```

```
data: {"type":"usage","usage":{"input_tokens":1000,"output_tokens":200}}
```

```
data: [DONE]
```

UI should show:

Thinking/searching state

Sources found

Answer streaming

Citations appearing

Final usage summary

21. Database Schema

users

users

- id
- email
- name
- created_at

workspaces

workspaces

- id
- name
- owner_user_id
- plan
- status
- created_at

workspace_members

workspace_members

- id
- workspace_id
- user_id
- role

api_keys

api_keys

- id
- workspace_id
- user_id
- name
- key_prefix
- key_hash
- environment
- status
- scopes
- allowed_models
- rpm_limit
- daily_request_limit
- monthly_budget_cents
- last_used_at
- created_at
- revoked_at

Important: store only the hash of the key, not the raw key.

requests

requests

- id
- workspace_id
- user_id
- api_key_id
- endpoint
- model
- status
- request_body_redacted
- response_body_redacted
- error
- created_at

usage_events

usage_events

- id
- request_id
- workspace_id
- user_id

- api_key_id
- model
- provider
- input_tokens
- output_tokens
- cached_tokens
- search_queries
- pages_fetched
- tool_calls
- provider_cost_usd
- billed_usd
- latency_ms
- status
- created_at

rate_limit_events

rate_limit_events

- id
- workspace_id
- api_key_id
- limit_type
- limit_value
- current_value
- blocked
- created_at

provider_routes

provider_routes

- id
 - brok_model
 - provider_name
 - provider_model
 - priority
 - is_active
 - input_cost_per_million
 - output_cost_per_million
 - created_at
-

22. API Key Generation Implementation

Key format

brok_sk_live_RANDOM
brok_sk_test_RANDOM

Example:

brok_sk_live_8fM2kL9xQp92s...

Storage rule

Do not store the full key.

Store:

key_prefix = brok_sk_live_8fM2
key_hash = sha256(full_key + secret_salt)

When request comes in:

incoming_key
→ hash it
→ compare to stored hash
→ authenticate

Key reveal rule

Show key only once after creation.

Dashboard later shows only:

brok_sk_live_.....f92a

23. Abuse Protection

Required protections

- Rate limits
- Budget limits
- IP throttling
- Key-level quotas
- Workspace-level quotas
- Max output token caps
- Max search depth based on plan
- Max concurrent requests
- Prompt size limits
- Request timeout
- Streaming timeout
- Automatic key pause on abuse

Dangerous patterns to block

- Infinite loops
- Very large max_tokens
- Repeated 500 errors
- Mass key generation
- Bot signup abuse
- Rapid free-tier account creation
- Requests from suspicious IP ranges

Admin abuse alert example

Alert: Workspace "acme-test" used 92% of monthly quota in 11 minutes.
Action suggested: throttle to 10 RPM or pause API key.

24. MVP Build Plan

Phase 1: Core Brok Chat

Build:

- Landing page
- Login/signup

Brok Chat UI
MiniMax backend call
Streaming response
Thread history
Basic usage tracking

Phase 2: Brok API Keys

Build:

API key creation
Key hashing
API gateway auth
/v1/chat/completions
Usage logging
Rate limits
Basic dashboard

Phase 3: Brok Playground

Build:

Model selector
Prompt editor
Streaming response
Request JSON viewer
Response JSON viewer
curl/JS/Python snippets
Usage estimate
Save/replay request

Phase 4: Brok Search API

Build:

Search provider integration
Source extraction
Citation generation
/v1/search/completions
Search usage tracking

Search depth limits

Phase 5: Admin Panel

Build:

Admin overview
User usage table
API key management
Provider routing
Cost dashboard
Abuse alerts
Rate limit overrides

Phase 6: Billing

Build:

Stripe integration
Plans
Usage caps
Monthly credits
Overage billing
Invoices
Usage alerts

25. Existing Admin Panel Additions

Add a new top-level nav item:

Brok API

Inside it:

Overview
API Keys
Usage

- Logs
- Rate Limits
- Providers
- Costs
- Abuse
- Playground
- Settings

Admin dashboard widgets

- Brok API Usage Today
- Brok Revenue Today
- Brok Provider Cost Today
- Brok Gross Margin
- Top 10 Users by Cost
- Top 10 Users by Requests
- Failed Requests
- Rate Limited Requests
- Suspicious Workspaces

Admin key management

Admin can:

- view all keys
- pause key
- revoke key
- rotate key
- edit scopes
- set custom limits
- view key logs

Admin provider management

Admin can:

- add provider
- edit provider model
- disable provider
- set fallback

change cost assumptions
set routing policy

26. UX Copy

Dashboard empty state

Create your first Brok API key

Use Brok's API to add search-powered AI, chat, and coding intelligence to your app. Your key is only shown once, so copy it somewhere safe.

API key warning

Keep this key secret. Do not expose it in frontend code, browser extensions, mobile apps, or public repositories.

Usage page headline

Understand every request

Track tokens, search calls, latency, cost, and usage across all your Brok API keys.

Playground headline

Test Brok before shipping

Try chat, search, code, and agent requests directly from your browser. Copy production-ready code when you are done.

27. Core Success Metrics

Product metrics

API keys created
First successful API request
Daily active developers
Requests per user
Playground runs
Docs-to-key conversion
Free-to-paid conversion

Business metrics

Revenue
Provider cost
Gross margin
Average cost per request
Average revenue per request
Monthly active workspaces
Churn

Reliability metrics

p50 latency
p95 latency
p99 latency
error rate
provider failure rate
rate limit hit rate
stream disconnect rate

28. Non-negotiable Guardrails

These are required before giving users API access:

Provider keys server-side only
API key hashing
Rate limits
Usage tracking
Monthly budget caps
Admin kill switch
Provider cost dashboard

Abuse detection
Key revocation
Request logging
Max output tokens
Search depth limits

Without these, one abusive user can destroy your provider bill.

29. The Clean Pitch

Use this positioning:

Brok is a low-cost intelligence API for search, chat, code, and agents.

It gives developers OpenAI-compatible endpoints, live web citations, streaming responses, API keys, usage tracking, and cheap MiniMax-powered reasoning behind one clean platform.

Avoid this positioning:

Brok is MiniMax but hidden behind my own key.

Better:

Brok abstracts model providers behind one developer-friendly API with routing, citations, usage tracking, rate limits, and billing.

That sounds like a real infrastructure company.

admin panel

adm **Brok Admin Panel PRD: View & Control Everything**

1. Goal

Build a single **Brok Admin Panel** where you can see and manage everything happening across the platform:

- Brok Search
- Brok App Builder
- Brok Presentations
- Brok API
- Users
- Costs
- Logs
- Models
- Providers
- Abuse
- Billing
- System Health

The admin panel should answer these questions instantly:

- Who is using Brok?
- What are they building/searching/generating?
- How much does it cost us?
- How much revenue are we making?
- Which requests failed?
- Which users are abusing the system?
- Which app projects were generated?
- Which presentations were created?
- Which API keys are active?
- Which provider/model is being used?
- Are we profitable?

Brok's app-builder direction is inspired by Dyad's local/open-source AI app-builder flow, while presentations are inspired by the uploaded Gamma-style [presentation-ai](#) reference, which supports outline-first deck generation, themes, editable slides, real-time generation, sharing, recording, and PPTX export.

2. Admin Panel Route

/admin

Subroutes:

/admin/overview
/admin/users
/admin/workspaces
/admin/search
/admin/app-builder
/admin/presentations
/admin/api
/admin/projects
/admin/logs
/admin/costs
/admin/billing
/admin/models
/admin/providers
/admin/rate-limits
/admin/abuse
/admin/health
/admin/settings

3. Admin Sidebar

Admin

- Overview
- Users
- Workspaces
- Brok Search
- App Builder
- Presentations
- API Platform
- All Projects
- Logs
- Costs

- Billing
- Models
- Providers
- Rate Limits
- Abuse Detection
- System Health
- Settings

For MVP:

Overview
Users
Search
App Builder
Presentations
API
Logs
Costs
Abuse
Providers

4. Admin Overview Page

Route

/admin/overview

Purpose

This is the mission control page.

It should show the whole Brok business in one screen.

Top KPI Cards

Total Requests Today
Active Users Today
Search Queries Today

Apps Generated Today
Presentations Created Today
Slides Generated Today
API Calls Today
Provider Cost Today
Revenue Today
Gross Margin
Error Rate
Avg Latency

Example:

Requests Today: 42,918
Active Users: 1,284
Apps Generated: 219
Presentations Created: 87
Slides Generated: 1,024
Provider Cost: \$31.42
Revenue: \$112.90
Gross Margin: 72.1%
Error Rate: 1.8%

Charts

Usage over time
Cost over time
Revenue over time
Search/App/Presentation/API split
Model usage split
Provider usage split
Error rate over time
Latency p50/p95/p99

Tables

Top Users by Usage
Top Users by Cost
Top Workspaces
Most Expensive Requests
Recent Errors
Abuse Alerts

Provider Health
Recent Generated Apps
Recent Presentations

5. Global Search Admin

Feature

At the top of the admin panel, add a universal search bar:

Search users, projects, presentations, API keys, logs, request IDs...

Admin can search:

user email
workspace name
API key prefix
project name
presentation title
request ID
error message
model name
provider name

Example:

Search: animesh@...

→ shows user profile, projects, API keys, presentations, logs, billing

6. Users Page

Route

/admin/users

Table Columns

User
Email
Plan
Workspace
Status
Searches Today
Apps Generated
Presentations Created
API Calls
Cost Today
Revenue
Margin
Risk Score
Last Active
Created At

User Detail Page

/admin/users/:userId

Tabs:

Overview
Usage
Searches
App Projects
Presentations
API Keys
Logs
Costs
Billing
Abuse
Settings

User Actions

Suspend user
Pause user
Change plan
Reset usage

Set custom limits
View all logs
View all generated content
Refund
Delete user
Mark as trusted

7. Workspaces Page

Route

/admin/workspaces

Table Columns

Workspace
Owner
Members
Plan
Searches
Apps
Presentations
API Calls
Monthly Cost
Monthly Revenue
Status
Created At

Workspace Detail Tabs

Members
Projects
Presentations
API Keys
Usage
Costs
Billing
Logs

8. Brok Search Admin

Route

/admin/search

Purpose

View every Perplexity-style search/query happening in Brok.

Table Columns

Time
User
Query
Mode
Sources Used
Citations
Model
Provider
Input Tokens
Output Tokens
Cost
Latency
Status

Search Detail Drawer

Clicking a query opens:

Original query
Resolved query
Generated search queries
Sources found
Sources selected

Sources rejected
Final answer
Citations
Follow-up suggestions
Model route
Token usage
Cost
Latency
Errors

Filters

Date range
User
Workspace
Search mode
Model
Provider
Status
Cost > X
Latency > X
Has error

Admin Actions

View answer
View sources
Block source domain
Boost source domain
Replay query
Debug citation quality
Mark bad answer
Refund usage

9. App Builder Admin

Route

/admin/app-builder

Purpose

View every Lovable/Dyad-style app project generated inside Brok.

The uploaded Dyad reference is an open-source local AI app builder positioned as an alternative to Lovable, v0, Bolt, and Replit, with local/private behavior and bring-your-own API keys. Brok's app-builder admin should monitor the equivalent generated projects, prompts, files, builds, previews, and costs.

Subpages

/admin/app-builder/projects
/admin/app-builder/generations
/admin/app-builder/files
/admin/app-builder/builds
/admin/app-builder/errors
/admin/app-builder/costs

9.1 App Projects Table

Columns:

Project
Owner
Workspace
Status
Build Status
Files
Generations
Preview URL
Tokens Used
Cost
Created
Last Updated

Statuses:

draft

generating
preview_ready
build_failed
exported
deleted
suspended

Actions:

Open project
View files
View generations
View build logs
View preview
Disable preview
Delete project
Suspend project
Refund generation

9.2 App Project Detail Page

Route:

/admin/app-builder/projects/:projectId

Tabs:

Overview
Prompts
Files
Diffs
Build Logs
Preview
Costs
Errors
Security

Overview

Show:

Project name
Owner
Initial prompt
Current status
Framework
Dependencies
Total files
Total generations
Last build status
Total cost
Created at
Updated at

Prompts Tab

Show every user prompt/edit:

Prompt
Timestamp
Files changed
Model used
Tokens
Cost
Build result

Example:

Prompt: "make it look more premium and add pricing"
Files changed: Hero.tsx, Pricing.tsx, App.tsx
Build: passed
Cost: \$0.014

Files Tab

Show project file tree:

package.json
src/
 App.tsx

components/
Hero.tsx
Pricing.tsx
Navbar.tsx

Admin can:

view file
copy file
view latest diff
view file history
restore previous version

Build Logs Tab

Show:

Build command
Install logs
TypeScript errors
Vite errors
Repair attempts
Final status
Duration

Preview Tab

Show iframe preview with warning:

Admin preview runs in sandboxed mode.

Security Tab

Show flags:

external scripts
suspicious links
credential forms
obfuscated code

dangerous browser APIs
hidden redirects

10. Presentations Admin

Route

/admin/presentations

Purpose

View all Gamma-style AI presentations generated in Brok.

The uploaded `presentation-ai` reference supports AI-powered deck generation, an outline-first workflow, editable outlines, 38 built-in themes, custom themes, PPTX theme import, real-time slide generation, presentation mode, public sharing, recording controls, and PowerPoint export.

Subpages

/admin/presentations/decks
/admin/presentations/generations
/admin/presentations/slides
/admin/presentations/themes
/admin/presentations/assets
/admin/presentations/exports
/admin/presentations/shares
/admin/presentations/costs

10.1 Decks Table

Columns:

Presentation
Owner
Workspace

Slides
Theme
Style
Language
Status
Public/Private
Exports
Cost
Created
Last Edited

Statuses:

outline_generating
outline_ready
slides_generating
editing
ready
exporting
exported
failed
shared
deleted

Actions:

Open deck
View outline
View slides
View generation logs
View exports
Disable public share
Delete deck
Refund generation

10.2 Deck Detail Page

Route:

/admin/presentations/:presentationId

Tabs:

Overview
Outline
Slides
Assets
Speaker Notes
Exports
Share Links
Generation Logs
Costs
Security

Overview

Show:

Title
Owner
Prompt/topic
Slide count
Theme
Style
Language
Web search enabled
Model used
Total tokens
Image calls
Export count
Share status
Total cost

Outline Tab

Show editable/generated outline:

Slide 1: Title
Slide 2: Problem
Slide 3: Solution
Slide 4: Market

...

Slides Tab

Show all slide thumbnails.

Each slide detail:

slide title
layout type
content JSON
speaker notes
assets used
generation status

Assets Tab

Show:

AI images
stock images
uploaded images
charts
media embeds
asset provider
cost
prompt used

Exports Tab

Show:

PPTX exports
PDF exports later
image exports
status
file URL
created at

Share Links Tab

Show:

share ID
visibility
views
created at
last viewed
disable link

11. API Platform Admin

Route

/admin/api

Purpose

View and manage Brok API usage and user-created API keys.

Subpages

API Keys
API Calls
Usage
Rate Limits
Logs
Costs
Docs Usage
Playground Runs

API Keys Table

Key Name
Prefix
Owner

Workspace
Environment
Scopes
Allowed Models
Status
Requests Today
Monthly Usage
Last Used
Created

Actions:

Pause key
Resume key
Revoke key
Rotate key
Edit limits
View logs

Never show raw API keys.

Only show:

brok_sk_live_8fM2.....9a21

12. All Projects Page

Route

/admin/projects

Purpose

A unified view of everything users create in Brok.

Project Types

Search Thread

App Project
Presentation Deck
API Playground Session
Shared Link
Exported File

Table Columns

Type
Name/Title
Owner
Workspace
Status
Cost
Created
Last Updated
Visibility
Actions

This lets you view everything without going into each module.

13. Global Logs Page

Route

/admin/logs

Purpose

View every important event across the platform.

Log Types

search_request
app_generation
app_build
presentation_outline_generation

presentation_slide_generation
api_call
api_key_created
export_created
share_link_created
billing_event
provider_error
abuse_event
admin_action

Table Columns

Time
Event Type
User
Workspace
Resource
Status
Model
Provider
Cost
Latency
Error
Request ID

Log Detail Drawer

Show:

request id
event type
user
workspace
resource id
provider route
input tokens
output tokens
cost
latency
status
error
redacted request JSON

redacted response JSON
timeline

Redaction Rules

Never show:

raw API keys
provider secrets
passwords
cookies
authorization headers
private tokens
OAuth credentials

14. Costs Page

Route

/admin/costs

Purpose

Make sure Brok does not lose money.

Top Cards

Provider Cost Today
Provider Cost This Month
Revenue Today
Revenue This Month
Gross Margin
Most Expensive User
Most Expensive Feature
Negative Margin Users

Cost Breakdown

Cost by feature
Cost by model
Cost by provider
Cost by user
Cost by workspace
Cost by project
Cost by presentation
Cost by API key

Feature Cost Split

Search cost
App generation cost
Presentation generation cost
Image generation cost
Web search cost
API usage cost
Export cost
Storage cost

Margin Table

User
Plan
Revenue
Provider Cost
Gross Margin
Feature Used Most
Risk

Alerts

User cost exceeds revenue
Provider cost spike
Image generation spike
Failed build loops causing excess cost
Presentation generation retries too high
API key abusing token output

15. Billing Admin

Route

/admin/billing

Table Columns

User
Workspace
Plan
MRR
Usage This Month
Overage
Payment Status
Renewal Date
Stripe Customer ID

Actions

Change plan
Apply coupon
Issue refund
Cancel subscription
Pause subscription
Set usage credits
Set hard cap
View invoices

Payment Statuses

active
trialing
past_due
canceled
unpaid
failed

16. Models Admin

Route

/admin/models

Purpose

Manage Brok's public/internal model names.

Table Columns

Brok Model
Used For
Provider
Provider Model
Enabled
Input Cost
Output Cost
Avg Latency
Error Rate
Plans Allowed

Example:

brok-search → MiniMax + Search
brok-app-builder → MiniMax/OpenCode route
brok-present → MiniMax + slide generator
brok-code → coding edits
brok-lite → cheap chat

Actions:

Enable/disable model
Change provider
Set fallback

Set max tokens
Set allowed plans
Set pricing multiplier

17. Providers Admin

Route

/admin/providers

Purpose

Manage MiniMax, search providers, image providers, export providers, and local model routes.

Provider Table

Provider
Type
Status
Requests Today
Cost Today
Avg Latency
Error Rate
Rate Limit Errors
Last Error

Provider types:

LLM
Search
Image
Stock Media
Export
Storage
Local Model

Actions:

Enable provider
Disable provider
Set fallback
Rotate provider key
Update pricing assumptions
Set timeout
Set retry policy

Provider secrets must be hidden.

18. Rate Limits Admin

Route

/admin/rate-limits

Purpose

Control usage by plan, user, workspace, feature, model, and API key.

Limit Types

requests per minute
requests per day
tokens per day
tokens per month
app generations per day
projects per user
presentation decks per month
slides per month
AI images per month
PPTX exports per month
API calls per minute
monthly dollar budget
max output tokens
max build repair attempts

Plan Defaults

Free

Search: 100/day
App projects: 3
App generations: 20/day
Presentations: 3/month
Slides: 30/month
API access: disabled or test only

Pro

Search: higher limit
App projects: more
App generations: 300/day
Presentations: 100/month
Slides: 1,000/month
API access: enabled

Team

Shared workspace
Higher limits
Team billing
Brand themes
Admin controls

19. Abuse Detection Admin

Route

/admin/abuse

Purpose

Catch users who can hurt cost, security, or reputation.

Abuse Signals

too many API requests
too many app generations
huge project files
repeated failed builds
phishing page generation
fake login page generation
credential harvesting
spam presentations
copyrighted brand clone requests
excessive AI image generation
many accounts from same IP
negative margin behavior

Abuse Table

Risk Score
User
Workspace
Feature
Reason
Cost Last Hour
Requests Last Hour
Status
Suggested Action

Risk Score

0-30 normal
31-70 watch
71-100 high risk

Actions:

Throttle user
Pause API key
Suspend project
Disable public share
Suspend workspace
Require payment method
Block IP

Mark safe

20. System Health Admin

Route

/admin/health

Services

API Gateway
Database
Redis
Queue Workers
Search Provider
MiniMax Provider
Image Provider
App Build Runtime
Presentation Export Worker
Storage
Auth
Billing Webhooks

Each service shows:

status
uptime
latency
error rate
last checked
last error

Statuses:

healthy
degraded
down
unknown

21. Admin Audit Logs

Every admin action must be logged.

Table

admin_audit_logs

- id
- admin_user_id
- action
- target_type
- target_id
- before_value
- after_value
- ip_address
- created_at

Examples:

Paused API key
Deleted presentation
Disabled public share
Suspended user
Changed provider route
Changed rate limit
Issued refund

22. Database Tables Needed

Core

users
workspaces
workspace_members
admin_audit_logs

Search

search_threads
search_messages
search_sources
search_citations
search_followups

App Builder

app_projects
app_project_files
app_generations
app_builds
app_versions
app_exports

Presentations

presentations
presentation_outlines
presentation_slides
presentation_themes
presentation_assets
presentation_generations
presentation_exports
presentation_shares

API

api_keys
api_requests
usage_events
rate_limit_events

Cost / Abuse

cost_events

billing_events
abuse_events
provider_events
system_health_events

23. Unified Usage Event Schema

Every action in Brok should create a usage event.

usage_events

- id
- user_id
- workspace_id
- feature
- resource_type
- resource_id
- event_type
- model
- provider
- input_tokens
- output_tokens
- search_queries
- image_calls
- build_seconds
- export_count
- provider_cost_usd
- billed_usd
- latency_ms
- status
- error_code
- created_at

Feature values:

search
app_builder
presentations
api
billing
admin

This is what lets the admin panel view everything in one place.

24. Admin Permissions

Roles:

owner
admin
support
finance
viewer

Owner

Everything

Admin

Manage users
Manage projects
Manage decks
Manage API keys
Manage providers
View costs
Change limits

Support

View users
View logs
View projects
Cannot see provider secrets
Cannot change billing/providers

Finance

View billing

View costs

Issue refunds

No content moderation access unless needed

Viewer

Read-only

25. MVP Admin Panel

Build this first:

1. /admin overview
2. users table
3. global logs
4. costs page
5. app builder projects page
6. presentation decks page
7. API keys page
8. abuse alerts
9. provider health

MVP Must-Have Cards

Requests today

Apps generated today

Presentations created today

Slides generated today

API calls today

Cost today

Revenue today

Gross margin

Errors today

Abuse alerts

MVP Must-Have Tables

- Recent users
- Recent app projects
- Recent presentations
- Recent API calls
- Recent errors
- Top users by cost

26. Admin Panel UI Style

Use a serious SaaS dashboard style.

Visual Direction

- dark sidebar
- light main workspace
- dense but clean tables
- KPI cards at top
- charts in middle
- tables below
- slide-over drawers for details
- colored status badges

References

- Stripe dashboard
- Vercel dashboard
- OpenAI usage dashboard
- Cloudflare analytics
- Linear admin views

Detail Pattern

Most rows should open a right-side drawer, not navigate away immediately.

Example:

Click app project
→ drawer opens
→ see owner, prompt, files, build logs, cost
→ click “Open full page” if needed

27. Final Product Definition

Brok Admin Panel is the internal command center for viewing and controlling every part of Brok.

It shows users, workspaces, searches, app projects, presentations, API keys, logs, costs, billing, providers, errors, abuse, and system health in one place.

The admin panel’s job is simple:

See everything.

Debug anything.

Control costs.

Stop abuse.

Keep Brok profitable and reliable.

app builder

Got it — **no agent builder**.

Brok should have a **Lovable/Dyad-style app builder**: user describes an app, Brok generates the app, previews it, lets the user edit through chat, and eventually deploy/export it.

The uploaded reference is Dyad, which describes itself as a **local, open-source AI app builder** similar to Lovable, v0, Bolt, and Replit, with local/private behavior and bring-your-own API keys.

<https://github.com/dyad-sh/dyad>

PRD: Brok App Builder

1. Product Goal

Build a new Brok feature that clones the **Lovable/Dyad app-building flow**, not an agent builder.

The feature should let users type:

build me a landing page for an AI search engine called Brok

or:

build me a SaaS dashboard with login, pricing, settings, and usage charts

Then Brok should:

- understand the request
- create an app plan
- generate files
- show a live preview
- let user edit by chatting
- show changed files
- run/build the app

fix errors automatically
allow export/deploy

The experience should feel like:

Lovable + Dyad + Bolt + Brok branding

Not:

n8n
agent workflow builder
automation builder
node canvas

2. Product Positioning

What Brok App Builder is

Brok App Builder is a chat-first AI app creation tool.

Users describe an app in natural language, and Brok turns it into a working frontend/backend project.

What it is not

It is not an agent builder.

Do not include:

workflow nodes
agent schedules
tool permissions
automation runs
agent deployment
n8n-style canvas

Instead, focus on:

app generation
file editing
live preview
chat-based iteration
error fixing
deployment/export

3. Route

Main route:

/app-builder

Local route:

localhost:8080/app-builder

Optional routes:

/app-builder/projects

/app-builder/:projectId

/app-builder/:projectId/files

/app-builder/:projectId/settings

For MVP:

/app-builder

4. Core User Flow

User opens /app-builder
→ sees one big chat input
→ describes app idea
→ Brok creates app plan
→ Brok generates files
→ live preview loads

- user chats edits
- Brok modifies files
- app rebuilds
- user sees changes instantly
- user exports/deploys project

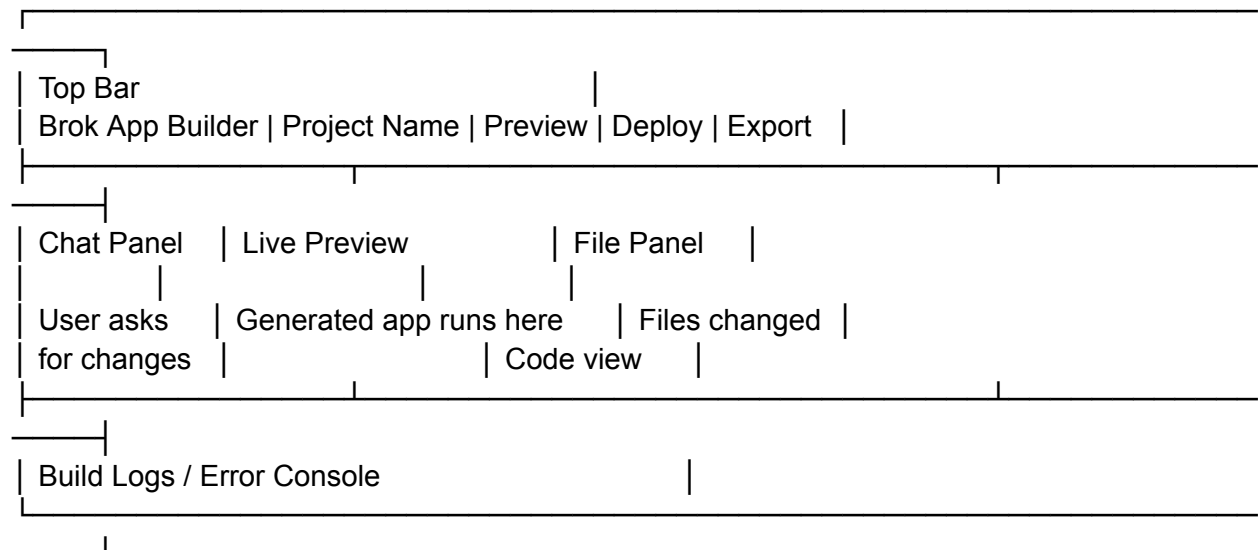
Core loop:

Prompt → Plan → Generate → Preview → Edit → Fix → Deploy

This is the entire product.

5. Main UI Layout

The app builder should have 4 main zones.



Left: Chat Panel

This is the main control surface.

User types:

make the hero more like Perplexity
add a pricing section
make it dark mode
add a dashboard page
fix the navbar on mobile

Brok responds with short build updates:

I updated the hero, added a pricing section, and adjusted the mobile navbar.

Center: Live Preview

This is the most important area.

It should show the generated app in an iframe or preview pane.

Preview modes:

desktop
tablet
mobile

Actions:

refresh preview
open in new tab
copy preview link
inspect selected element later

Right: File Panel

Shows generated files and changes.

Example:

Files changed

- |— src/App.tsx
- |— src/components/Hero.tsx
- |— src/components/Pricing.tsx
- |— src/components/Navbar.tsx
- |— package.json

Clicking a file opens code view/diff.

Bottom: Build Logs

Shows build status, errors, and fixes.

Example:

Generating project...
Installing dependencies...
Building preview...
Preview ready.

If error:

Build failed: missing import in Pricing.tsx
Fixing automatically...
Build passed.

6. Homepage / Empty State

When user opens `/app-builder`, show a clean Lovable-style start screen.

Layout

Brok App Builder

What do you want to build?

[Describe your app...]

Example chips:

- Landing page for an AI startup
- SaaS dashboard with usage analytics
- Portfolio website
- AI search engine homepage
- Admin panel for API usage
- Marketplace app

Input behavior

The input should support:

plain text prompt
paste website idea
paste design notes
upload image later
upload files later

MVP only needs text.

7. App Generation Flow

When user submits a prompt, Brok should run this process:

1. Interpret app request
2. Decide project type
3. Create implementation plan
4. Generate file tree
5. Generate code
6. Install dependencies if needed
7. Build app
8. Show preview
9. Save project
10. Suggest next edits

Example

User:

build me a landing page for Brok, an AI search engine

Brok creates:

Project: Brok Landing Page

Files:

- package.json
- src/App.tsx
- src/components/Navbar.tsx
- src/components/Hero.tsx
- src/components/HowItWorks.tsx
- src/components/Pricing.tsx
- src/components/Footer.tsx

Preview shows the app.

Follow-up suggestions:

- Add pricing
- Make it more like Perplexity
- Add animated search examples
- Create a dashboard page
- Make it mobile responsive

8. App Planning System

Before code generation, Brok should generate a hidden/internal app plan.

App plan structure

```
{
  "project_name": "Brok Landing Page",
  "project_type": "landing_page",
  "style": "minimal, dark, AI-search inspired",
  "pages": [
    {
      "name": "Home",
      "path": "/",
      "sections": ["Navbar", "Hero", "Features", "How It Works", "Pricing", "Footer"]
    }
  ],
  "components": [
    "Navbar",
    "Hero",
```

```
"FeatureGrid",
"SearchDemo",
"Pricing",
"Footer"
],
"dependencies": ["react", "tailwindcss", "lucide-react"],
"notes": [
  "Use clean Perplexity-style layout",
  "Use Brok branding",
  "Prioritize fast preview"
]
}
```

The user does not need to see this by default, but they can open “Plan.”

9. File Generation System

Brok needs a virtual project filesystem.

Project structure

```
brok-projects/
  project_123/
    package.json
    index.html
    src/
      main.tsx
      App.tsx
      components/
        Navbar.tsx
        Hero.tsx
        Pricing.tsx
      styles/
        globals.css
```

Database/project storage

Store:

project id
user id
project name
prompt history
file tree
file contents
preview status
build status
last generated version
created at
updated at

10. Chat-Based Editing

This is the core Lovable-style interaction.

User should be able to say:

make the hero more premium
add a waitlist form
make the pricing cards cleaner
change the accent color to blue
add a sidebar dashboard
make it responsive
fix the broken button

Brok should:

understand requested change
identify relevant files
modify only needed files
show changed files
rebuild preview
summarize what changed

Good response

Updated the hero, waitlist form, and pricing cards. The preview is ready.

Bad response

Here is the full code you can copy.

The app builder should actually modify the generated project, not just dump code into chat.

11. Follow-Up Suggestions

After every generation or edit, Brok should suggest useful next actions.

Example after landing page generation

Try next:

- Add a pricing page
- Make the design more like Perplexity
- Add a dashboard preview section
- Add login/signup buttons

Example after dashboard generation

Try next:

- Add mock API usage charts
- Add a settings page
- Add dark mode
- Add billing table

Follow-ups should be short, clickable chips.

Clicking a chip should send it as the next chat message.

12. Live Preview PRD

Goal

The preview should update after each successful build.

Preview states:

empty
generating
building
ready
failed

Preview toolbar

Desktop | Tablet | Mobile
Refresh
Open
Copy link

Build failed state

If build fails, center panel should show:

Preview failed to build.

Error:
Cannot find module './Pricing'

[Fix with Brok]

But Brok should auto-fix most build errors without requiring the user to click.

13. Error Fixing Loop

Brok needs an automatic repair loop.

Flow

1. Generate/edit files
2. Run build

3. If build passes, show preview
4. If build fails, read error
5. Ask model to patch relevant files
6. Rebuild
7. Repeat up to 3 times
8. If still failing, show error clearly

Console log example

Build failed: Navbar.tsx imports missing Button component.
Applying fix...
Rebuilding...
Build passed.

This is crucial for making the app builder feel magical.

14. Code View / Diff View

Right panel should support:

Files
Changed
Diff
Code
Dependencies

File tree

src/
 App.tsx
 components/
 Hero.tsx
 Pricing.tsx
package.json

Diff view

Show before/after changes after each prompt.

Example:

Hero.tsx

+ added animated search bar

+ updated headline

+ changed CTA button

Actions

copy file

revert file

revert last change

download project

15. Project History

Every prompt/edit should create a version.

Version record

version id

prompt

files changed

timestamp

build status

summary

User can:

view history

restore version

compare versions

For MVP, keep basic:

undo last change

restore previous version

16. Deploy / Export

MVP export

Allow user to download project as ZIP.

Export ZIP

Later deploy

Deploy to:

Vercel

Netlify

Cloudflare Pages

Brok hosting

Deploy modal

Deploy Brok Landing Page?

Build status: Passing

Pages: 1

Last updated: 2 minutes ago

[Deploy]

17. Project Dashboard

Route:

/app-builder/projects

Shows all generated apps.

Project card

Project name

Preview thumbnail

Last edited

Build status

Open

Export

Delete

MVP can skip this if `/app-builder` only handles one active project, but you will eventually need it.

18. Integration into Brok

Add App Builder to Brok navigation.

Main nav

Brok

- Search
- App Builder
- API
- Library
- Settings

Admin nav

Admin

- Overview
- Users
- API Usage
- App Builder
- Costs
- Logs
- Providers
- Settings

19. Admin Panel for App Builder

Add a new admin section:

Admin → App Builder

Subpages:

Projects
Generations
Build Logs
Costs
Errors
Abuse
Templates

Admin Projects

Columns:

Project
Owner
Status
Files
Build Status
Tokens Used
Cost
Created
Last Updated

Actions:

view project
disable project
delete project
view logs
view cost

Admin Generations

Each prompt/edit generation should be logged.

Columns:

Time
User
Project
Prompt
Files Changed
Tokens
Cost
Build Result

Admin Build Logs

Show:

project id
build command
status
duration
error message
repair attempts

Admin Costs

Track:

tokens per generation
provider cost
build/runtime cost
storage cost
total project cost
user revenue
margin

Admin Abuse

Flag:

too many projects created
huge prompts
repeated failed builds
crypto/mining app attempts
malicious code generation
spam sites
phishing pages
credential-stealing pages

20. Rate Limits

App builders can get expensive, so add hard limits.

Limits to track

projects per user
generations per day
edits per project per day
max files per project
max file size
max total project size
max build attempts
max repair attempts
max tokens per generation
max monthly cost

Suggested free limits

Free:

- 3 projects
- 20 generations/day
- 50 files/project
- 3 repair attempts/generation
- export only

Suggested pro limits

Pro:

- unlimited projects within fair use
 - 300 generations/day
 - 250 files/project
 - deploy/export
 - higher model limits
-

21. Templates

Brok should have templates to make generation better.

Template categories

Landing Page
SaaS Dashboard
Admin Panel
Portfolio
Marketplace
AI Chat App
Documentation Site
Waitlist Page
Internal Tool

Template flow

User can start from:

blank prompt
template
uploaded screenshot later
existing GitHub repo later

MVP:

blank prompt + templates

22. App Builder Models

Internally use MiniMax/OpenCode style routing.

Model routes

simple edit → fast cheap model

large app generation → stronger coding route

bug fix → coding repair route

planning → planner model

Suggested internal roles

Planner

- turns prompt into app plan

Coder

- generates/edits files

Reviewer

- checks for missing imports/build issues

Repairer

- fixes build errors

This can all be powered behind the scenes. User just sees “Brok is building.”

23. Build Runtime

For MVP, generate React + Vite apps.

Why Vite

fast

simple

easy preview

works well with generated React

simple dependency handling

MVP stack

React
TypeScript
Vite
Tailwind
lucide-react
shadcn-style components if available

Build commands

npm install
npm run dev
npm run build

For preview, use sandboxed iframe.

24. Security Requirements

Generated apps must run safely.

Sandbox rules

run preview in iframe
disable dangerous browser APIs if possible
block external script injection by default
do not expose Brok secrets
do not expose MiniMax keys
do not allow server-side code execution in MVP
scan generated code for suspicious patterns

Block or warn on

phishing pages
credential harvesting
malware

crypto miners
fake login clones
hidden redirects
obfuscated scripts
exfiltration code

25. MVP Scope

Build this first:

1. /app-builder route
2. Prompt input
3. App planning
4. File generation
5. File tree
6. Live preview
7. Chat edits
8. Build logs
9. Auto-fix build errors
10. Export ZIP
11. Admin logs/cost tracking

Do not build yet:

agent workflows
node canvas
scheduled automations
full backend generation
database provisioning
complex deployment
GitHub import
browser agent

26. Best Demo Flow

Use this demo:

Step 1

User opens:

localhost:8080/app-builder

Step 2

User types:

build a landing page for Brok, an AI search engine that feels like Perplexity but cheaper and faster

Step 3

Brok shows:

Planning app...
Generating files...
Building preview...
Preview ready.

Step 4

Preview shows:

Brok landing page
hero section
search demo
features
pricing
footer

Step 5

User types:

make it look more premium and add a dashboard preview section

Step 6

Brok edits files, rebuilds, and preview updates.

Step 7

User types:

add pricing cards and make the CTA say start building

Step 8

Preview updates.

Step 9

User clicks:

Export ZIP

This proves the whole app builder.

27. Final Product Definition

Brok App Builder is a Lovable-style AI app builder inside Brok.

Users describe an app, Brok plans it, generates the project files, runs a live preview, lets users edit through chat, fixes build errors, tracks file changes, and exports/deploys the final app.

It is not an agent builder. It is a chat-first app builder.

The product should feel like:

“I type what I want, and Brok builds the app in front of me.”

